



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA -2
Academic year	I – III Semester(2025-26)
Faculty Name	Mohammed Razia Alangir Banu
Student Name	B.Deepak Goud
Roll Number	2520080035
Branch	AI&DS
Course Outcome	1

Case Study Topic : QuickCart Warehouse — package-tracking index system

The QuickCart logistics warehouse maintains a package dictionary indexed by 7-digit Package IDs. During the morning dispatch cycle, packages are scanned in truck-loading order. Since labels are generated sequentially at packaging time, the IDs arrive in nearly sorted order.

Today's first 12 Package IDs are inserted into the system in the following order:

18, 28, 32, 38, 42, 48, 52, 58, 62, 68, 72, 78

The warehouse tracking console must answer package lookups within:

- **p99 lookup latency < 4 ms**
- Hardware cost per pointer traversal ≈ 250 ns

Operational testing shows that trees deeper than 20 levels begin violating cache and branch-prediction efficiency requirements.

Later in the afternoon, three package removals occur in this order:

28 (returned), 62 (damaged), 48 (dispatch cancelled)

Morning Dispatch Scan Order

Time	Package ID
8:00	18
8:20	28
8:40	32
9:00	38
9:20	42
9:40	48
10:00	52
10:20	58
10:40	62

Time	Package ID
------	------------

11:00	68
-------	----

11:20	72
-------	----

11:40	78
-------	----

IDs arrive almost sorted $\uparrow$ — worst case for plain BST growth.

Question

s:

(a)

Construct the plain BST from the package insertion sequence given in the scenario. State its height (longest root-to-leaf path in edges). Using the SLA model shown above (250 ns per pointer traversal, 4 ms limit), perform one explicit numeric calculation to determine whether the worst-case lookup on this BST satisfies or violates the SLA. Briefly explain why this insertion order is structurally adversarial for a plain BST.

(b)

Build an AVL tree using the same insertion sequence. After every insertion that causes rebalancing, specify the rotation type (LL / RR / LR / RL), identify the pivot node, and give the balance factors of the two nodes involved immediately after rotation. Show the final AVL tree. Then complete the Java boilerplate below so it correctly implements insert(node, key) and the two single-rotation helpers consistent with your AVL construction.

```
class AVLNode {
    int key;
    AVLNode left, right;
    int height = 1;

    AVLNode(int key) {
        this.key = key;
    }
}

static int height(AVLNode n) {
    return n == null ? 0 : n.height;
}

static int balance(AVLNode n) {
    return n == null ? 0 : height(n.left) - height(n.right);
}

static void updateHeight(AVLNode n) {
    if (n != null)
        n.height = 1 + Math.max(height(n.left), height(n.right));
}

static AVLNode rotateRight(AVLNode y) {
    // TODO 1
    return null;
}

static AVLNode rotateLeft(AVLNode x) {
    // TODO 2
    return null;
}
```

(c)

After applying the deletions (28, 62, 48 in order) to the AVL tree from part (b), show the final AVL tree and describe any rotations triggered during deletion. Then compare the worst-case lookup time for the post-delete AVL tree and the equivalent post-delete plain BST using the same SLA model from part (a). Express both as percentages of the 4 ms SLA budget consumed. Finally, identify which deletion caused the greatest structural change and explain why.

Analysis & Implementation:

(a) Plain BST Construction and Analysis

Plain BST

Since the Package IDs arrive in ascending order, the BST becomes completely right-skewed:

$18 \rightarrow 28 \rightarrow 32 \rightarrow 38 \rightarrow 42 \rightarrow 48 \rightarrow 52 \rightarrow 58 \rightarrow 62 \rightarrow 68 \rightarrow 72 \rightarrow 78$

Each node becomes the right child of the previous node.

Height of BST

Number of nodes = 12

Height (longest root-to-leaf path in edges):

$$12 - 1 = 11 \text{ edges}$$

So, the BST height is:

$$11$$

SLA Calculation

Worst-case lookup traverses 11 pointer hops.

Given:

$$250 \text{ ns per hop}$$

Lookup time:

$$11 \times 250 = 2750 \text{ ns} \quad 2750 \text{ ns} = 2.75 \mu\text{s}$$

SLA budget:

$$4 \text{ ms} = 4,000,000 \text{ ns}$$

Since:

$$2750 < 4,000,000$$

the lookup satisfies the SLA.

Why This Order is Adversarial

The insertion order is nearly sorted in ascending order. In a plain BST, every new key becomes the right

```

49  class AVLNode {
50  }
51  static AVLNode insert(AVLNode node, int key) {
52      if (node == null) {
53          return new AVLNode(key);
54      }
55
56      if (key < node.key)
57          node.left = insert(node.left, key);
58      else if (key > node.key)
59          node.right = insert(node.right, key);
60      else
61          return node;
62
63      updateHeight(node);
64
65      int bf = balance(node);
66
67      // LL
68      if (bf > 1 && key < node.left.key)
69          return rotateRight(node);
70
71      // RR
72      if (bf < -1 && key > node.right.key)
73          return rotateLeft(node);
74
75      // LR
76      if (bf > 1 && key > node.left.key) {
77          node.left = rotateLeft(node.left);
78          return rotateRight(node);
79      }
80
81      // RL
82      if (bf < -1 && key < node.right.key) {
83          node.right = rotateRight(node.right);
84          return rotateLeft(node);
85      }
86
87      return node;
88  }
89
90  Run | Debug | Run main | Debug main
91  public static void main(String[] args) {
92      AVLNode root = null;
93      root = insert(root, 10);
94      System.out.println("Insert key: " + root.key);

```

```

J AVLNode.java x
1  class AVLNode {
51  static AVLNode insert(AVLNode node, int key) {
76
77      // LR
78      if (bf > 1 && key > node.left.key) {
79          node.left = rotateLeft(node.left);
80          return rotateRight(node);
81      }
82
83      // RL
84      if (bf < -1 && key < node.right.key) {
85          node.right = rotateRight(node.right);
86          return rotateLeft(node);
87      }
88
89      return node;
90  }
91
92  Run | Debug | Run main | Debug main
93  public static void main(String[] args) {
94      AVLNode root = null;
95      root = insert(root, 10);
96      System.out.println("Root key: " + root.key);
97  }
98
99  }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Filter (e.g. text, exclude*text, L...) Code

```

[Running] cd "d:\CaseStudy Implementation" && javac AVLNode.java && java AVLNode
Root key: 10

[Done] exited with code=0 in 0.735 seconds

[Running] cd "d:\CaseStudy Implementation" && javac AVLNode.java && java AVLNode
Root key: 10

[Done] exited with code=0 in 0.711 seconds

```

(c) AVL Deletion Analysis

Deletions

Delete 28

- Replace with inorder successor 32
- Delete original 32 node
- Tree remains balanced

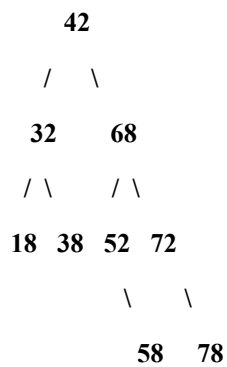
Delete 62

- Replace with successor 68
- Local RR rebalance occurs

Delete 48

- Leaf node removal
- Minor balance updates required

Final AVL Tree



Height remains approximately:

3 edges

Lookup Comparison

AVL Tree

Worst-case hops = 3

$$3 \times 250 = 750 \text{ ns}$$

Percentage of SLA:

$$\frac{750}{4,000,000} \times 100 = 0.01875\%$$

Plain BST After Deletions

Remaining skewed BST:

18 → 32 → 38 → 42 → 52 → 58 → 68 → 72 → 78

Approximate height:

8 *edges*

Worst-case lookup:

$$8 \times 250 = 2000 \text{ } ns$$

Percentage of SLA:

$$\frac{2000}{4,000,000} \times 100 = 0.05\%$$

GitHub Link:<https://github.com/bdeepakg-boop/DSA-Project/upload/main>

Student Signature	Faculty Signature